1. Static/dynamic operations **03A1**

我们知道，视其是否会改变数据集的内容及结构，对数据结构的操作可分为静态、动态两种。

a) 然而如果数据集中各元素的逻辑结构与物理结构未必一致，此时的静态、动态操作又当如何区分？

b) 你能举出这类数据结构的一例吗？

2. Call-by-rank/position **03A2**

a) 秩与位置之间有何联系与区别？

b) 阅读示例代码，了解重载操作符“[]”、“+”和“-”各自的功能；

c) 这些操作符可以精简代码，但我们为何不建议经常使用？

3. Distance Between List Nodes **03A2**

任给同一列表中的两个节点，我们需要通过如下接口，得到它们之间的距离：

```
template <typename T> Rank distance( LNP<T> p, LNP<T> q );
```

当然，时间复杂度必须为$\mathcal{O}(d)$，其中d就是返回的距离。

a) 试确认：若已知在列表中，p位于q之前，则借助本节重载的减号操作符不难实现distance()；

b) 如果二者的前后次序未知呢？

4. List[-k] **03A2**

在一个由单向链表实现、也不记录总体长度的列表L中，为找到倒数第k个节点L[-k]，有以下两种方法：

- 先经过一次遍历，统计出n = L.size；然后再次遍历，找到节点L[n-k]。

- 使用间距为k的两个位置指针slow和fast，同步地遍历列表，待到fast抵达tail时，返回slow。

不难看出，两种方法的渐进复杂度同阶，但实际的运行时间却可能差异很大。

试通过实验就此做一测试和统计，并做出解释。

5. Sentinel **03B[1-4]**

a) 引入哨兵head、tail之后，列表各主要接口的代码逻辑在哪些方面更为精简了？

b) 运行时间方面，这些接口各有多大的改进？

c) 试针对本节介绍的各种列表接口验证，借助哨兵的确可以正确处理各种边界情况（比如空表）。

6. insert()/remove() **03B1**

本节实现的节点插入、删除算法中，都有若干对pred、succ的调整。这些调整操作的次序是否可以调换？

7. movePred()/moveSucc() **03B1**

调用调用算法movePred(s)时，若当前节点this恰好就是节点s，结果会如何？

8. copy() **03B2**

a) 如果在List::copy()中用insertFirst()替换insertLast()，如何才能同样地完成列表的复制，而且新列表中的节点保持原先的相对次序？

b) 如此调整之后，算法的时间、空间性能有何变化？

9. Destructor **03B2**

本节给出了双向列表的析构算法，时间复杂度为$\mathcal{O}(n)$。

然而，如果是单向列表（只设有succ引用，而没有pred），该算法需要如何调整？复杂度还是$\mathcal{O}(n)$吗？

10. dedup() **03B3**

a) 试验证：该算法在最好情况下只需$\mathcal{O}(n)$时间；

b) 我们记得，Vector::dedup()算法总是需要$\mathcal{O}(n^2)$时间，为什么在这方面列表会不同于向量？

c) 试验证：该算法对find()首次调用时，查找区间总是为空；尽管如此，find()亦能正确处理；

d) 该算法的循环不变性为“始终维护一个无重的前缀”。如果改为后缀，又当如何实现该算法？效率如何？

e) 该算法在前缀中一旦发现与节点p相等的q，便随即删除q——如果反过来删除p，代码需如何调整？

f) 自行设计若干测例，验证该算法的正确性，尤其能正确处置各种边界情况；

g) 试按照其它思路，设计和实现这个接口，并做实测对比。

11. traverse(): Basic **03B4**

a) 对照向量遍历接口的运用示例，理解列表遍历接口的运用技巧；

b) 除了课上举过的示例，你还通过遍历实现过哪些算法功能？

12. Find & Search By Traversal **03B4**

我们在本章看到，无论无序列表还是有序列表，其查找算法的实质过程，无非都是对列表的遍历。

试照此理解，基于List::traverse()接口来实现List::find()及List::search()算法。

13. uniquify() **03C**

a) 该算法起始处，为何需要对平凡的列表（ _size < 2 ）做特判？

b) 你觉得可以如何改写，使得这段代码的逻辑更为简洁？

c) 自行设计若干测例，验证该算法的正确性，尤其能正确处置各种边界情况；

d) 试按照其它思路，设计和实现这个接口，并做实测对比。

14. search(): failure cases **03C**

a) 试确认，在查找失败时，该算法将返回查找区间的头“哨兵”h；

b) 试确认，对于任何元素e，在调用“insert(search(h, e, t), e)”之后，区间(h,t)依然有序；

c) 如果只是单纯的查找，如何根据search(h, e, t)的返回值，判断区间(h,t)内是否存在元素e呢？

15. search(): efficiency **03C**

a) 试确认，即便列表已经有序，课上实现的search()算法在最坏情况下仍需$\Omega(n)$时间；

b) 可否改写这个算法，降低其在最坏情况下的复杂度？

c) 如果可以，试写出算法；否则，试说明原因。

16. selectionSort() by swap **03D1**

本节的selectionSort()算法通过交换操作，将前缀中的最大节点m直接转入有序的后缀。

a) 试验证：也可以先经remove()从前缀中删除x，再通过insert()将其插入后缀；

相对于以上交换法，这可称作平移法。

b) 为何我们还是倾向于使用交换法？

17. getMax() **03D1**

为保证选择排序的稳定性，当前缀中的最大元素不唯一时，getMax()算法应当返回其中的最靠后者。

a) 试验证：getMax()算法的确是如此处置的；

b) 试验证：区间内的每一组相等元素，在getMax()的一趟执行过程中至多只会引发最大节点x的一次更新；

c) 如果将扫描的次序由自后向前改为自前向后呢？

d) 如果自前向后扫描，在最坏情况下最大节点记录需要更新多少次？试给出这样的一个实例。

18. Vector::selectionSort **03D1**

a) 试基于向量实现就地的选择排序算法，要求与列表一样，虽然仍需做$\mathcal{O}(n^2)$次比较，但仅需做$\mathcal{O}(n)$次移动；

b) 从实例代码包中找到已经实现了的Vector::selectionSort()，与你的实现做一比较。

19. Swap vs. Cycle **03D2**

a) 在一个序列中交换一对元素，序列中的哪些轮换会有变化？

b) 变化可能有几种情况？试逐一列举并说明。

20. Expected #Cycle By Exchanges **03D2**

考查对长度为n的随机序列做选择排序，约定采用交换法。

按照我们的惯例，在每一步迭代中，无序前缀里的最大元素与末元素分别记作**m**和**x**。

a) 试证明：若m和x不是同一节点，则在二者交换之前，它们应当同属一个轮换，且在其中紧邻；

b) 试证明：二者交换之后，该轮换的长度缩减一个单位，而m自成一个轮换；

c) 不难看出，当m和x系同一节点时，二者的交换是多余的。

试证明：如果输入序列包含k个轮换，那么这种情况在其选择排序过程中会恰好出现k次；

d) 试证明：对于任何一步迭代，若当时无序前缀的长度为r，则发生这种情况的概率为$1/r$；

e) 试证明：采用交换法的选择排序过程中，期望地会出现$\mathcal{O}(\log n)$次原地的交换；

f) 试证明：在长度为n的随机序列中，期望地含有$\mathcal{O}(\log n)$个轮换。

21. Expected #Cycle By Moves **03D2**

考查改用平移法的选择排序算法。

c') 是否也有类似上一题中c)的结论？

亦即，对于包含c个轮换的输入序列，选择排序中恰好出现c次原地的移动？

如果是，试给出证明；否则请举出反例；

e') 是否也有类似与上一题e)的结论？

亦即，在选择排序过程中，期望地会出现$\mathcal{O}(\log n)$次多余的移动？

如果是，试给出证明；否则请举出反例；

22. insertionSort() vs. selectionSort() **03E1**

a) 我们知道插入排序属于在线算法，你知道还有哪些排序算法也是？

b) 玩纸牌时若希望手中的牌始终顺序排列，你在抓牌过程中为何会采用插入排序而非其它算法？

c) 我们知道插入排序属于输入敏感算法，你知道还有哪些排序算法也是？

d) 不难验证，选择排序既非在线的，也没有最好情况。

那么反过来，它相对于插入排序有何优势？

23. Vector::insertionSort() **03E1**

a) 试基于向量实现就地的插入排序算法；

b) 你能否做到与列表一样，虽然仍需做$\mathcal{O}(n^2)$次比较，但仅需做$\mathcal{O}(n)$次移动？

如果能，如何做到？否则，为什么不能？

24. insertionSort() by move() **03E1**

本节的insertionSort()算法，直接通过moveSucc()接口将节点x插入有序的前缀。

a) 试验证：也可以先经remove()从后缀中删除x，再通过insert()将其插入前缀；

b) 为何我们还是倾向于使用讲义中的方法？

25. Best/average case of insertionSort() **03E1**

a) 我们知道，当输入序列业已有序时插入排序只需运行$\mathcal{O}(n)$时间，这种最好情况出现的概率有多大？

b) 试验证：新元素插入有序前缀之后，有可能发现其实是原地未动。

在整个排序过程中，这种情况期望地会出现多少次？

c) 平均而言，插入排序需要运行多少时间？

26. search() in insertionSort() **03E1**

我们知道，插入排序算法在插入每个新元素之前，都要在有序前缀中查找确定合适的位置。

a) 既然这里的前缀始终都是有序的，为何不将顺序查找改作二分查找？

b) 如果必须改为二分查找，存储有序前缀的数据结构需要哪些接口？各接口的效率需要有多高？

27. #inversion **03E2**

a) 试举例说明，一个长度为n序列，最多可嫩 · 含有$\Omega(n^2 gl)$个逆序对；

b) 就数学期望而言，一个长度为n的序列中会含有多少个逆序对？

c) 根据上一结论，插入排序算法的平均时间复杂度应是多少？

28. #inversion **03E2**

设一个长度为$2n$的向量，用02D节bubblesort()算法的提前终止版，外循环仅迭代了nh步便完成了排序。

a) 该向量最少、最多可能包含多少个逆序对？

b) 若$nl = 15$，试分别给出一个满足上述条件的整数向量。

29. Inversion & Exchange **03E2**

a) 在序列中交换一对逆序元素，逆序对的总数会如何变化？可能有哪几种情况？

b) 如果可以选择，在排序过程中我们为何倾向于优先交换相距更远的逆序对？

c) 试证明：一个序列中的逆序对无论多少，我们都只需$\mathcal{O}(n)$次交换便可使之有序；

d) 在你所知的排序算法中，哪些可以保证所做的交换操作不超过$\mathcal{O}(n)$次？哪些不能保证？

30. Complexity of 2-way merge **03F1**

a) 如果待归并的列表长度不等甚至相差悬殊，二路归并算法的运行时间还能保持线性吗？

b) 一般地，若二者的长度分别为n和m，运行时间应是多少？

31. 2-way merge by move() **03F1**

本节实现的List::merge()算法，直接通过movePred()接口将节点x移到y之前。

a) 试验证：也可以先经remove()从子列表中摘除x，再通过insert()将其归入最终的列表；

b) 为何我们还是倾向于使用讲义中的方法？

32. Vector/List::mergeSort() **03F1**

我们看到，基于向量或列表都可以实现归并排序，但两种方式各有什么优势与不足？

33. Vector/List::mergeSort() **03F1**

我们知道，无论是基于向量或列表来实现归并排序，只要如讲义那样始终以中点来切分子任务（子序列），就能保证总体的时间复杂度为$\mathcal{O}(n \log n)$f。

a) 如果改为总是按$2/3 : 1/3$的比例来切分呢？

b) 改用$15/16n : 1/16$的比例呢？

c) 一般地，改用$\lambda : 1-\lambda$的比例呢？

d) 如果总是按$n-1 : 1$的比例来切分，那么mergeSort()相当于退化成了哪种排序算法？

34. Counting inversions **03F2**

a) 试确认，借助归并排序必能在$\mathcal{O}(n \log n)$时间内统计出任一序列中的逆序对总数；

b) 你还有什么其它办法也可以完成这一统计？

c) 有没有更加高效的办法？

35. Cursor list **03XA**

a) 在什么情况下，我们需要以游标模式来实现（或者更准确地，来模拟）列表？

b) 对讲义中的实例，理解这种结构的原理，推敲其运作过程，并尝试实现该结构。

36. Java Vector/List **03XB**

a) 了解Java语言中interface、implements等关键词的含义与作用；

b) 在示例代码包中找到用Java描述和实现的数据结构及算法；

c) 阅读其中Vector和List的代码，与C++的实现做一对比；

d) 运行自带的测试程序，理解测试过程及结果。

37. Python List **03XC**

a) 在示例代码包中找到用Python描述和实现的数据结构及算法；

b) 阅读源文件reverse.py，理解其中那两个reverse()算法的原理；

c) 了解这里所用的测试方法，运行并观察统计结果；

d) 两个reverse()算法的效率相差很多，试就此给出解释。